

Classi e oggetti

Classi e oggetti

Introduzione alla programmazione orientata agli oggetti in Python.

Qual è la funzione di classi e oggetti

Immagina di dover gestire le informazioni di 30 studenti in un programma. Senza un modo per organizzare i dati, ti ritroveresti con decine di variabili sparse ovunque: `nome1` , `eta1` , `voto1` , `nome2` , `eta2` , `voto2` ...

Le **classi** risolvono questo problema: ti permettono di raggruppare dati e funzioni correlate in un unico "pacchetto" chiamato **oggetto**.

La differenza tra classe e oggetto

Una **classe** è come il **progetto di un'automobile**: descrive come deve essere fatta, ma non è un'auto vera.

Un **oggetto** è l'**automobile vera**, costruita seguendo quel progetto.

Dal stesso progetto (classe) puoi costruire molte auto (oggetti) diverse — ognuna con il suo colore, la sua targa, il suo chilometraggio.

Definire una classe

```
class Persona:  
    pass    # "pass" significa: classe vuota per ora, aggiungeremo cose dopo
```

Il costruttore: `__init__`

Il metodo `__init__` è il **costruttore** — viene eseguito automaticamente ogni volta che crei un nuovo oggetto. Serve a impostare i dati iniziali dell'oggetto.

```
class Persona:
    def __init__(self, nome, eta):
        # "self" è l'oggetto che stiamo creando
        # self.nome e self.eta sono i dati dell'oggetto (chiamati
        "attributi")
        self.nome = nome
        self.eta = eta
```

`self` è un riferimento all'oggetto stesso. È sempre il primo parametro di ogni metodo in una classe, anche se quando chiami il metodo non lo scrivi esplicitamente.

Creare oggetti (istanze)

```
alice = Persona("Alice", 16)  # Crea una Persona con nome="Alice" e eta=16
bob = Persona("Bob", 17)     # Crea una Persona diversa

# Accedere ai dati dell'oggetto con il punto
print(alice.nome)  # Alice
print(bob.eta)     # 17
```

`alice` e `bob` sono due **istanze** diverse della stessa classe `Persona`. Hanno gli stessi "campi", ma valori diversi — come due auto dello stesso modello ma con targhe diverse.

Aggiungere metodi

I **metodi** sono funzioni definite dentro la classe. Descrivono le **azioni** che un oggetto può fare.

```

class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def saluta(self):
        # Questo metodo usa i dati dell'oggetto (self.nome, self.eta)
        print(f"Ciao, mi chiamo {self.nome} e ho {self.eta} anni.")

    def compleanno(self):
        # Questo metodo modifica un dato dell'oggetto
        self.eta += 1
        print(f"Buon compleanno {self.nome}! Ora hai {self.eta} anni.")

alice = Persona("Alice", 16)
alice.saluta()      # → Ciao, mi chiamo Alice e ho 16 anni.
alice.compleanno()  # → Buon compleanno Alice! Ora hai 17 anni.
alice.saluta()      # → Ciao, mi chiamo Alice e ho 17 anni.

```

Attributi di classe vs attributi di istanza

- **Attributi di istanza:** appartengono al singolo oggetto (ogni oggetto ha il suo valore)
- **Attributi di classe:** sono condivisi da tutti gli oggetti della classe (come una bacheca comune)

```

class Studente:
    scuola = "Liceo Scientifico"  # Attributo di classe: uguale per tutti

    def __init__(self, nome):
        self.nome = nome          # Attributo di istanza: diverso per
ciascuno

alice = Studente("Alice")
bob = Studente("Bob")

print(alice.scuola)  # Liceo Scientifico
print(bob.scuola)    # Liceo Scientifico ← stesso valore per tutti
print(alice.nome)    # Alice
print(bob.nome)      # Bob                ← valore diverso per ciascuno

```

Il metodo `__str__` : come stampare un oggetto

Per default, se fai `print(alice)` Python stampa qualcosa di poco leggibile come `<__main__.Persona object at 0x...>`. Il metodo speciale `__str__` ti permette di decidere tu come appare l'oggetto quando viene stampato.

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def __str__(self):
        # Questo viene usato da print() e str()
        return f"Persona(nome={self.nome}, eta={self.eta})"

alice = Persona("Alice", 16)
print(alice)  # → Persona(nome=Alice, eta=16)
```

Esempio completo: la classe Rettangolo

```
class Rettangolo:
    def __init__(self, base, altezza):
        self.base = base
        self.altezza = altezza

    def area(self):
        # Calcola l'area: base × altezza
        return self.base * self.altezza

    def perimetro(self):
        # Calcola il perimetro: 2 × (base + altezza)
        return 2 * (self.base + self.altezza)

    def __str__(self):
        return f"Rettangolo {self.base}x{self.altezza}"

r = Rettangolo(5, 3)
print(r)           # → Rettangolo 5x3
print(r.area())    # → 15
print(r.perimetro()) # → 16

# Puoi creare più rettangoli indipendenti
r2 = Rettangolo(10, 4)
print(r2.area())   # → 40
```

Ogni rettangolo è un oggetto indipendente: cambiare `r` non influisce su `r2`.